

Module 2: Transport Layer and Network Layer



Module 2: Transport Layer and Network Layer

Transport Layer

- Services and Protocols
- UDP and TCP

Hands-on

- Sockets Introduction
- Elementary TCP Sockets
- TCP Client/Server Example
- I/O Multiplexing
 - select()
 - poll()
- Elementary UDP Sockets
- Advanced I/O Functions

Network Layer

- Introduction
- Network-layer Protocols
- Unicast Routing
- Multicast Routing
- Multicasting Basics
- Intra-domain Routing
- Inter-domain Routing
- Next Generation IP (IPv6)
- Quality of Service (QoS)

Hands-on

- Linux Routing Table
- Routing Caches
- Routing Cache Implementation
- Adding Routes using ip

Transport-Layer Protocols

Introduction

A transport-layer protocol is created by combining different transport-layer services such as reliability, flow control, error control, multiplexing, and congestion control.

Purpose

- Provide end-to-end communication between applications.
- Combine transport-layer services based on application requirements.
- Form the basis for practical protocols such as TCP and UDP.

Key Point

Different applications require different combinations of transport-layer services.

Evolution of Transport Protocols

Learning Approach

- 1 Begin with the simplest transport protocol.
- 2 Gradually introduce additional services.
- 3 Combine these services to form more sophisticated protocols.
- 4 Understand how TCP/IP transport protocols are designed.

Why This Approach?

Studying simple protocols first makes it easier to understand the operation of complex protocols such as TCP.

Simplex and Full-Duplex Communication

Simplex Communication

- Data flows in only one direction.
- Sender transmits.
- Receiver only receives.
- Used initially to explain protocol behavior.

Full-Duplex Communication

- Data flows in both directions.
- Both hosts can send and receive simultaneously.
- Used in practical transport protocols.

Exam Point

Transport-layer protocols are first studied as **simplex protocols** for simplicity and are later extended to **full-duplex protocols** used in real networks.

Simple Protocol

Definition

The **Simple Protocol** is the most basic transport-layer protocol.

Characteristics

- Connectionless protocol
- No Flow Control
- No Error Control
- Unidirectional (Simplex)
- Receiver is assumed to process packets immediately

Key Assumption

The receiver is never overwhelmed by incoming packets.

Working of the Simple Protocol

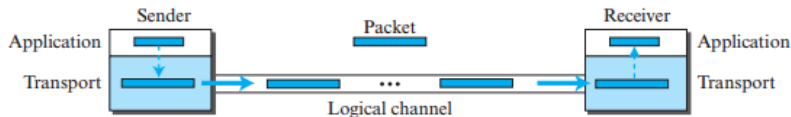
Sender Side

- 1 Receives a message from the Application Layer.
- 2 Encapsulates the message into a packet.
- 3 Sends the packet to the receiver.

Receiver Side

- 1 Receives the packet from the Network Layer.
- 2 Decapsulates the packet.
- 3 Delivers the message to the Application Layer.

Figure 3.17 Simple protocol



Finite State Machines (FSM)

Finite State Machine (FSM)

Both sender and receiver have only one state called the **Ready State**.

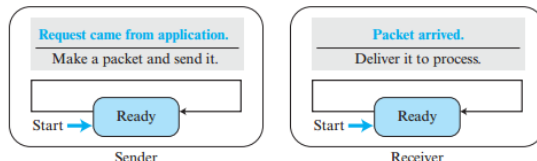
Sender FSM

- Wait for message
- Encapsulate message
- Send packet
- Return to Ready state

Receiver FSM

- Wait for packet
- Decapsulate packet
- Deliver message
- Return to Ready state

Figure 3.18 FSMs for the simple protocol

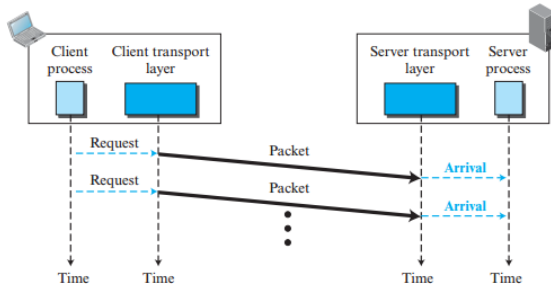


Example of Communication

Example 3.3

- Sender transmits packets continuously.
- Sender does not wait for any acknowledgement.
- Sender does not check receiver status.
- Receiver accepts packets immediately.

Figure 3.19 Flow diagram for Example 3.3



Simple Protocol – Key Takeaways

Summary

- Connectionless
- No Flow Control
- No Error Control
- Simplex Communication
- Sender always Ready
- Receiver always Ready
- One-State FSM
- Continuous Packet Transmission

Relation to UDP

The textbook states that the **UDP protocol is a slight modification of this Simple Protocol.**

Stop-and-Wait Protocol

Definition

The **Stop-and-Wait Protocol** is a **connection-oriented transport-layer protocol** that provides both **Flow Control** and **Error Control**.

Main Features

- Connection-oriented communication
- Sliding Window size = 1
- One packet is sent at a time
- Sender waits for ACK before sending next packet
- Uses checksum to detect errors

Key Point

Only one data packet and one acknowledgment can exist in the channel at any instant.

Working of Stop-and-Wait Protocol

Sender Operation

- 1 Receive message from Application Layer.
- 2 Encapsulate the message into a packet.
- 3 Add sequence number and checksum.
- 4 Send the packet.
- 5 Start the timer.
- 6 Wait for acknowledgment (ACK).

Receiver Operation

- 1 Receive packet.
- 2 Verify checksum.
- 3 Deliver correct packet to Application Layer.
- 4 Send acknowledgment.

Error Detection and Recovery

Error Detection

- Each packet contains a checksum.
- Receiver verifies checksum after packet arrival.

If Packet is Correct

- Deliver message to Application Layer.
- Send ACK.

If Packet is Corrupted

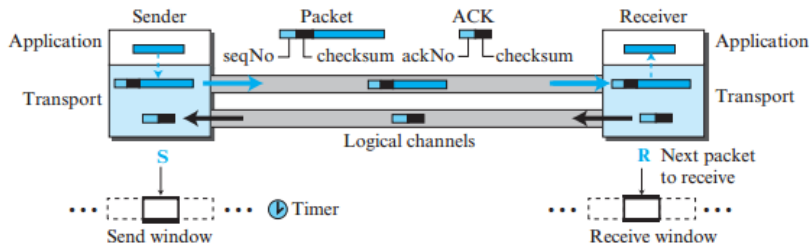
- Packet is silently discarded.
- No ACK is sent.
- Sender timeout expires.
- Sender retransmits the same packet.

Exam Point

Silence from the receiver indicates that the packet or ACK was lost or corrupted.

Protocol Operation

Figure 3.20 *Stop-and-Wait protocol*



Important Observations

- Sender keeps a copy of every transmitted packet.
- Timer starts immediately after transmission.
- ACK stops the timer.
- Timeout causes retransmission.
- Only one outstanding packet exists.

Sequence Numbers

Why Sequence Numbers?

Sequence numbers prevent duplicate packet delivery.

Three situations may occur:

- 1 Packet arrives successfully.
- 2 Packet is lost or corrupted.
- 3 ACK is lost or corrupted.

Observation

- Receiver must distinguish between
 - New packet
 - Retransmitted packet

Key Point

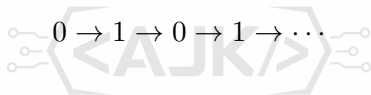
Only two sequence numbers are sufficient.

Modulo-2 Sequence Numbers

Why Only Two Sequence Numbers?

The Stop-and-Wait protocol allows only one outstanding packet at a time. Hence, only two sequence numbers are required.

Sequence Pattern



Modulo-2 Arithmetic

- Packet numbers alternate between 0 and 1.
- No need for sequence number 2.
- Eliminates ambiguity caused by retransmissions.

Exam Point

Stop-and-Wait uses **Modulo-2 Arithmetic** for sequence numbering.

Acknowledgment Numbers

Acknowledgment Rule

The ACK number always indicates the **next packet expected by the receiver**.

Received Packet	ACK Sent
0	ACK 1
1	ACK 0

Control Variables

- Sender Window Pointer : **S**
- Receiver Window Pointer : **R**
- Both windows have size = 1.

Key Point

ACK always announces the sequence number of the next expected packet.

Sender Finite State Machine (FSM)

Initial State

- Sender starts in the Ready State.
- Sender variable

$$S = 0$$

Ready State

- 1 Receive message from Application Layer.
- 2 Create packet.
- 3 Set sequence number = S .
- 4 Store a copy.
- 5 Send packet.
- 6 Start timer.
- 7 Move to Blocking State.

Sender FSM (Blocking State)

Blocking State Events

1 Correct ACK arrives

- Stop timer
- Slide window

$$S = (S + 1) \bmod 2$$

- Return to Ready State

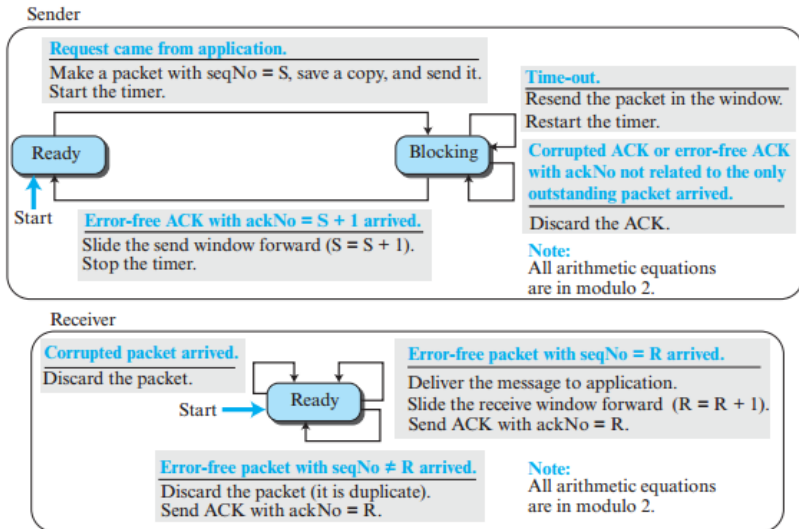
2 Corrupted/Wrong ACK

- Discard ACK

3 Timeout

- Retransmit stored packet
- Restart timer

Figure 3.21 *FSM for the Stop-and-Wait protocol*



Receiver Finite State Machine (FSM)

Receiver Characteristics

- Receiver is always in the Ready State.
- Receiver variable

$$R = 0$$

Receiver Events

- 1 Correct packet with



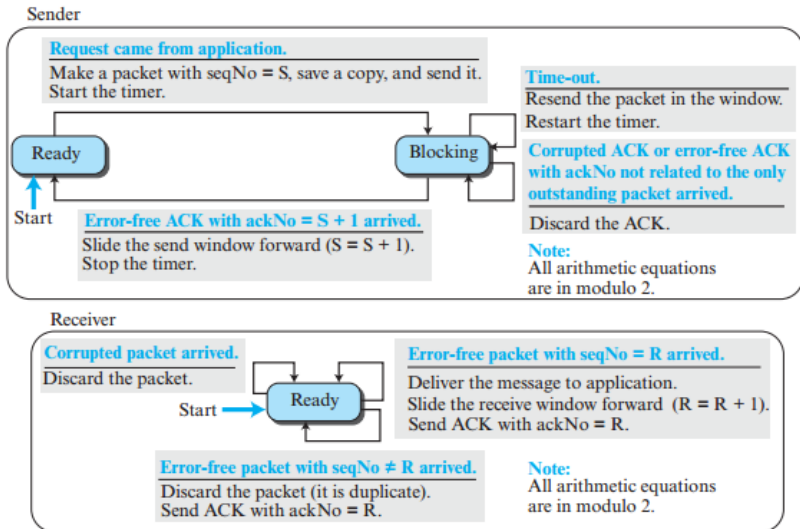
$$seqNo = R$$

- Deliver message.
- Slide window

$$R = (R + 1) \bmod 2$$

- Send ACK(R).
- 2 Duplicate packet
 - Discard packet.
 - Send ACK(R).

Figure 3.21 *FSM for the Stop-and-Wait protocol*



Receiver Processing

- Waits for incoming packet.
- Checks checksum for errors.
- Verifies expected sequence number.
- Sends acknowledgment after processing.

Receiver Actions

- Correct packet → Deliver message + Send ACK
- Duplicate packet → Discard + Send ACK
- Corrupted packet → Discard silently

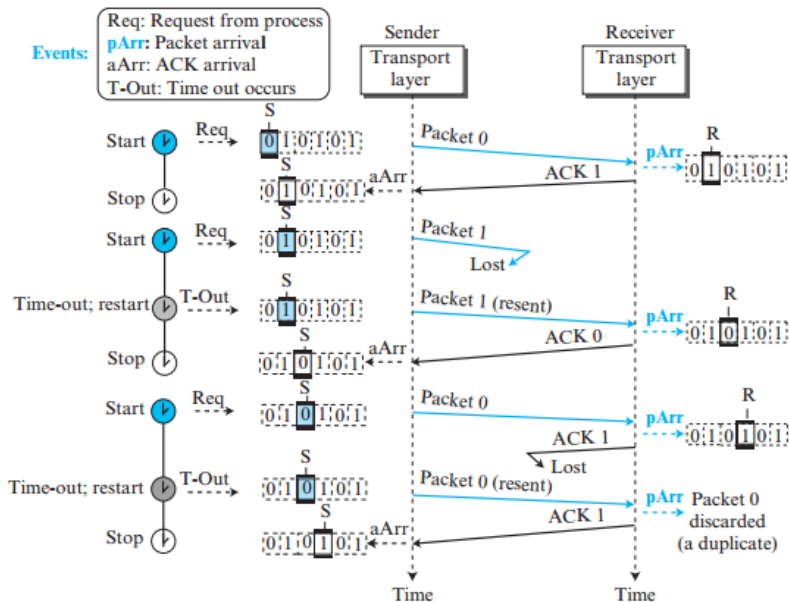
Example 3.4 – Stop-and-Wait Communication

Communication Example

The following events occur during transmission:

- 1 Packet 0 is transmitted successfully.
- 2 ACK 1 is received.
- 3 Packet 1 is lost.
- 4 Sender timeout expires.
- 5 Packet 1 is retransmitted.
- 6 ACK 0 is received.
- 7 Packet 0 is transmitted.
- 8 ACK is lost.
- 9 Sender retransmits Packet 0.
- 10 Receiver detects duplicate Packet 0 and sends ACK again.

Figure 3.22 Flow diagram for Example 3.4



Example 3.4 Illustration

Observation

- Lost packets are recovered using timeout.
- Lost acknowledgments result in retransmission.
- Duplicate packets are recognized using sequence numbers.

Efficiency of Stop-and-Wait Protocol

Bandwidth–Delay Product (BDP)

$$\text{BDP} = \text{Bandwidth} \times \text{Round Trip Delay}$$

It represents the number of bits that can be transmitted while waiting for an ACK.

Example 3.5

- Bandwidth = 1 Mbps
- RTT = 20 ms
- BDP =

$$1 \times 10^6 \times 20 \times 10^{-3} = 20,000 \text{ bits}$$

- Packet size = 1000 bits
- Utilization

$$\frac{1000}{20000} \times 100 = 5\%$$

Example 3.6

If 15 packets are transmitted before waiting for ACKs,

$$\text{Utilization} = \frac{15000}{20000} \times 100 = 75\%$$

Pipelining

- Stop-and-Wait does **not** support pipelining.
- Sender must wait for an ACK before sending the next packet.
- Later protocols (Go-Back-N and Selective Repeat) support pipelining by allowing multiple outstanding packets.

Go-Back-N (GBN) Protocol

Definition

Go-Back-N (GBN) is a **connection-oriented sliding window protocol** that improves transmission efficiency by allowing multiple packets to be sent before receiving acknowledgments.

Characteristics

- Connection-oriented protocol
- Provides Flow Control and Error Control
- Multiple outstanding packets allowed
- Receiver window size = 1
- Sender stores copies of transmitted packets

Key Point

Several data packets and acknowledgments can exist simultaneously in the channel.

Why Go-Back-N?

Problem with Stop-and-Wait

- Sender waits after every packet.
- Communication link remains idle.
- Poor utilization on high bandwidth-delay links.

Solution

- Allow multiple packets to be transmitted.
- Keep the communication channel busy.
- Wait for acknowledgments after sending several packets.
- Improve throughput.

Exam Point

GBN improves efficiency by filling the communication pipe with multiple packets.

Sender

- Can transmit several packets continuously.
- Stores copies of all outstanding packets.
- Waits for cumulative acknowledgments.
- Retransmits packets after timeout.

Receiver

- Can buffer only one packet.
- Accepts packets only in order.
- Discards out-of-order packets.
- Sends cumulative acknowledgments.

Sequence and Acknowledgment Numbers

Sequence Numbers

Sequence numbers use

$$\text{Modulo } 2^m$$

where m is the number of bits in the sequence number field.

Acknowledgment Numbers

ACKs are **cumulative**.

If

$$ackNo = 7$$

then packets

$$0, 1, 2, 3, 4, 5, 6$$

have been received successfully, and the receiver expects packet

Send Window

Send Window

The **Send Window** is an imaginary sliding window that contains the sequence numbers of packets that:

- Have been sent but not yet acknowledged.
- Can be sent immediately.

Maximum Window Size

$$2^m - 1$$

where

- m = number of bits in the sequence number field.

Key Point

The sender can have multiple outstanding packets inside the send window.

Regions of the Send Window

The send window divides the sequence number space into **four regions**.

① Acknowledged Packets

- Already acknowledged.
- Sender removes stored copies.

② Outstanding Packets

- Already transmitted.
- ACK not yet received.

③ Packets Ready to Send

- Inside the window.
- Waiting for application data.

④ Packets Outside Window

- Cannot be transmitted until the window slides.

Send Window Variables

Three variables define the position of the send window.

Variable	Meaning
Sf	First outstanding packet
Sn	Next packet to send
S_{size}	Size of the send window

Functions

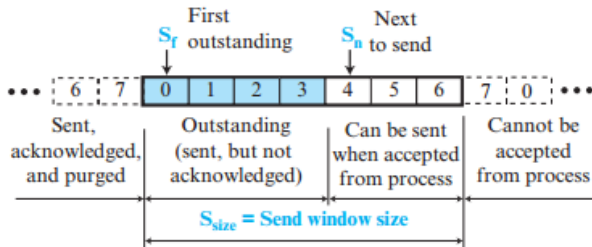
- Sf points to the oldest unacknowledged packet.
- Sn indicates the sequence number of the next packet.
- S_{size} remains fixed in GBN.

Exam Point

The send window is completely defined using Sf , Sn , and S_{size} .

Sliding Send Window

Figure 3.24 *Send window for Go-Back-N*



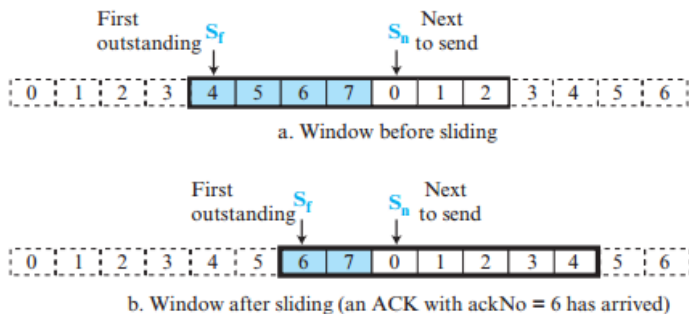
Window Sliding When an ACK is received,

$$S_f \leftarrow ackNo$$

The sender slides the window forward and new sequence numbers become available for transmission.

Sliding Mechanism

Figure 3.25 *Sliding the send window*

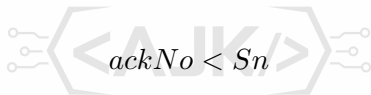


Sliding Mechanism

The send window slides when an error-free ACK satisfies

$$ackNo \geq Sf$$

and


$$ackNo < Sn$$

(using modulo arithmetic).

Result

- Previously acknowledged packets leave the window.
- New packets enter the window.
- Sender continues transmission without waiting for every ACK.

Receive Window

Receive Window

Unlike the sender, the receiver in Go-Back-N maintains a **receive window of size 1**.

Characteristics

- Accepts only the expected packet.
- Discards all out-of-order packets.
- Sends cumulative acknowledgments.
- Window slides one position after receiving the expected packet.

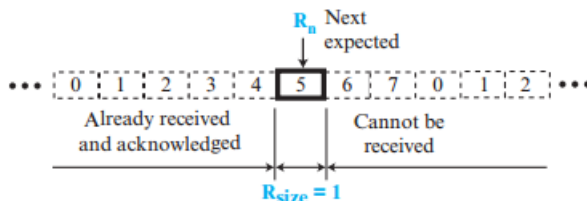
$$Rn = (Rn + 1) \bmod 2^m$$

Key Point

Only one variable, **Rn**, defines the receive window.

Receive Window Operation

Figure 3.26 *Receive window for Go-Back-N*



Receiver Actions

- Expected packet arrives → Accept and ACK.
- Out-of-order packet arrives → Discard and resend ACK.
- Corrupted packet arrives → Discard.

Timers and Retransmission

Timer

Although every transmitted packet could have its own timer, Go-Back-N uses only **one timer**.

Reason

- Timer belongs to the oldest outstanding packet.
- It always expires first.
- Simplifies implementation.

Go-Back-N Retransmission

When the timer expires, the sender retransmits **all outstanding packets**.

Why is it called Go-Back-N?

Suppose

$$Sf = 3, \quad Sn = 7$$

Then packets

are outstanding.

If the timer expires,

- Sender goes back to packet 3.
- Retransmits packets

3, 4, 5, 6

- Receiver discards duplicates already received.

Exam Point

On timeout, the sender goes back **N packets** and retransmits all

Finite State Machines (FSM)

Sender FSM

- Ready State
- Blocking State
- Uses variables

S_f, S_n

- Starts/stops timer
- Slides send window

Receiver FSM

- Always Ready
- Uses variable

R_n

- Accepts expected packet
- Discards others
- Sends cumulative ACK

Figure 3.27 FSMs for the Go-Back-N protocol

Sender

Note:

All arithmetic equations are in modulo 2^m .

Time-out.

Resend all outstanding packets.
Restart the timer.

Request from process came.

Make a packet ($seqNo = S_n$).
Store a copy and send the packet.
Start the timer if it is not running.
 $S_n = S_n + 1$.

Time-out.

Resend all outstanding packets.
Restart the timer.

A corrupted ACK or an error-free ACK with $ackNo$ outside window arrived.

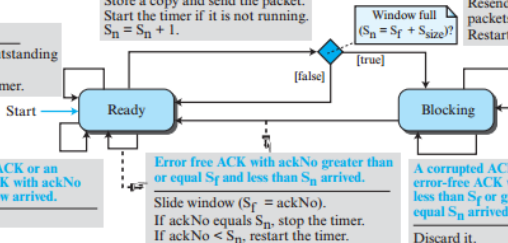
Discard it.

Error free ACK with $ackNo$ greater than or equal S_f and less than S_n arrived.

Slide window ($S_f = ackNo$).
If $ackNo$ equals S_n , stop the timer.
If $ackNo < S_n$, restart the timer.

A corrupted ACK or an error-free ACK with $ackNo$ less than S_f or greater than or equal S_n arrived.

Discard it.



Receiver

Note:

All arithmetic equations are in modulo 2^m .

Error-free packet with $seqNo = R_n$ arrived.

Deliver message.
Slide window ($R_n = R_n + 1$).
Send ACK ($ackNo = R_n$).

Corrupted packet arrived.
Discard packet.



Error-free packet with $seqNo \neq R_n$ arrived.

Discard packet.
Send an ACK ($ackNo = R_n$).

Initial State

$$Sf = Sn = 0$$

Ready State

- Receive request from Application Layer.
- Create packet with sequence number Sn .
- Store a copy of the packet.
- Send the packet.
- Start timer (if not already running).
- Increment

$$Sn = (Sn + 1) \bmod 2^m$$

- Move to Blocking State if window becomes full.

Blocking State

- Wait for ACK or Timeout.
- New packets cannot be transmitted if the window is full.

Receiver FSM and Window Size

Receiver Events

- 1 Correct packet
 - Deliver message.
 - Slide window

$$Rn = (Rn + 1) \bmod 2^m$$

- Send ACK(Rn).
- 2 Out-of-order packet
 - Discard packet.
 - Send duplicate ACK(Rn).
- 3 Corrupted packet
 - Discard packet.

Important Rule

Send Window Size $< 2^m$

Receive Window Size is always

Why Send Window Size Must be Less than 2^m

Case 1: Window Size = $2^m - 1$

- Lost ACKs cause retransmission.
- Receiver correctly detects duplicate packets.

Case 2: Window Size = 2^m

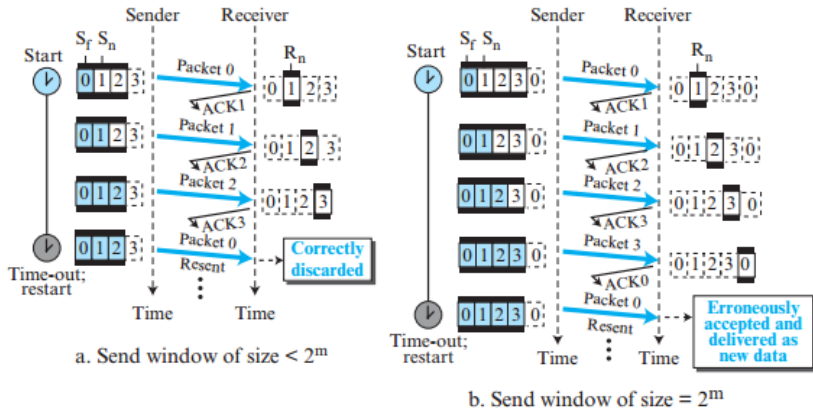
- Duplicate packet appears as a new packet.
- Receiver accepts an incorrect packet.
- Communication error occurs.

Exam Point

To avoid ambiguity,

$$\text{Window Size} < 2^m$$

Figure 3.28 *Send window size for Go-Back-N*



Examples 3.7 and 3.8

Example 3.7

- Reliable forward channel.
- ACK 2 is lost.
- ACK 3 is cumulative.
- No timeout occurs.
- Demonstrates cumulative acknowledgments.

Example 3.8

- Packet 1 is lost.
- Packets 2 and 3 arrive out of order.
- Receiver discards packets.
- Sends duplicate ACK1.
- Timeout causes retransmission of packets 1,2,3.

Figure 3.29 Flow diagram for Example 3.7

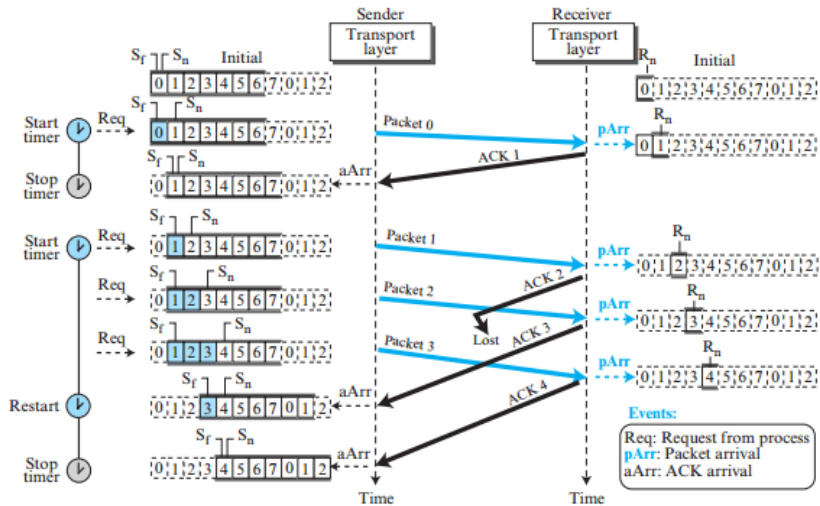
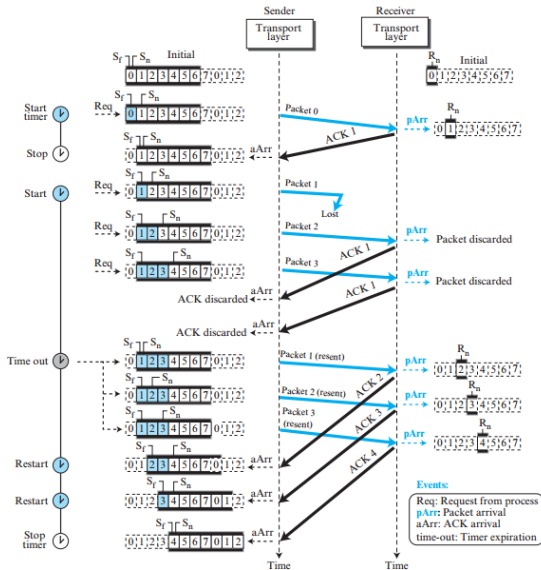


Figure 3.30 Flow diagram for Example 3.8



Go-Back-N vs Stop-and-Wait

Stop-and-Wait

- Window Size = 1
- One outstanding packet
- Wait after every packet
- Low throughput
- Modulo-2 numbering

Go-Back-N

- Window Size = $2^m - 1$
- Multiple outstanding packets
- Cumulative ACKs
- Better throughput
- Retransmits all outstanding packets

Key Takeaways

- Sliding Window Protocol
- Flow Control and Error Control
- Cumulative ACKs
- Send Window Size $< 2^m$
- Receive Window Size = 1

Selective Repeat (SR) Protocol

Definition

Selective Repeat (SR) is a connection-oriented sliding window protocol that improves efficiency by retransmitting only the packets that are lost or corrupted.

Characteristics

- Connection-oriented protocol
- Flow Control and Error Control
- Sender retransmits only erroneous packets
- Receiver buffers out-of-order packets
- Send and Receive windows have the same size

Key Point

Unlike Go-Back-N, SR stores out-of-order packets instead of discarding them.

Go-Back-N vs Selective Repeat

Go-Back-N

- Receiver window = 1
- Discards out-of-order packets
- Retransmits all outstanding packets
- Cumulative ACK

Selective Repeat

- Receiver window > 1
- Buffers out-of-order packets
- Retransmits only lost packets
- Individual ACKs

Exam Point

Selective Repeat improves bandwidth utilization by avoiding unnecessary retransmissions.

Send Window

Send Window

Maximum send window size:

$$2^{m-1}$$

where

m = number of bits in the sequence number field.

Example

If

$$m = 4$$

then

Sequence Numbers = 0 to 15

Maximum Window Size

Receive Window

Unlike Go-Back-N,
the receive window size is

$$2^m - 1$$

which is equal to the send window size.

Receiver

- Accepts out-of-order packets.
- Buffers received packets.
- Delivers packets to Application Layer only in order.
- Waits until missing packets arrive.

Key Point

Packets are never delivered out of order to the Application Layer.

Figure 3.32 *Send window for Selective-Repeat protocol*

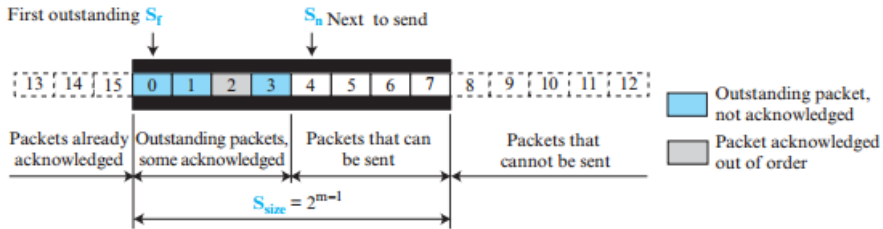
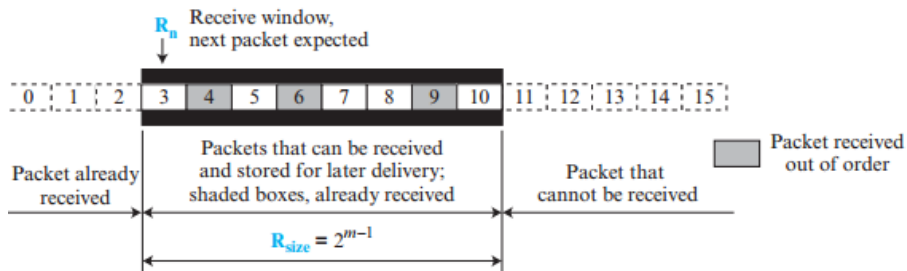


Figure 3.33 *Receive window for Selective-Repeat protocol*



Selective Repeat Windows

Observation

- Sender and receiver windows have equal size.
- Receiver buffers multiple packets.

Timer in Selective Repeat

Timer

Theoretically, Selective Repeat uses **one timer for each outstanding packet**.

Operation

- Every transmitted packet has its own timer.
- If a timer expires, only the corresponding packet is retransmitted.
- Unlike Go-Back-N, outstanding packets are treated individually.

Practical Implementation

- Most transport-layer implementations use only a single timer for simplicity.

Exam Point

GBN retransmits all outstanding packets, whereas SR retransmits only the timed-out packet.

Acknowledgments in Selective Repeat

Acknowledgment Semantics

In Selective Repeat, each acknowledgment refers to **one specific packet**.

Selective Repeat

means

- Packet n has been received correctly.
- No information is provided about other packets.

Go-Back-N

means

- All packets before n have been received.
- Receiver expects packet n next.

Example 3.9

Scenario

Sender transmits packets

0, 1, 2, 3, 4, 5

Later, the sender receives

$ACK = 3$

Interpretation

Go-Back-N

- Packets 0,1,2 received.
- Receiver expects packet 3.

Selective Repeat

- Only packet 3 is confirmed.
- Nothing is implied about other packets.

Key Difference

GBN acknowledgments are cumulative.

Initial State

$$Sf = Sn = 0$$

Ready State

- 1 Receive request from Application Layer.
- 2 Create packet using sequence number Sn .
- 3 Store a copy.
- 4 Send packet.
- 5 Start timer (if not running).
- 6 Increment

$$Sn = (Sn + 1) \bmod 2^m$$

- 7 Move to Blocking State if window becomes full.

Sender FSM (ACK Processing)

When an ACK arrives,

- Identify the corresponding outstanding packet.
- Mark only that packet as acknowledged.
- If


$$ACK = Sf$$

the send window slides.

- Continue sliding while consecutive packets are acknowledged.

Important

Unlike Go-Back-N, acknowledgments do not automatically confirm earlier packets.

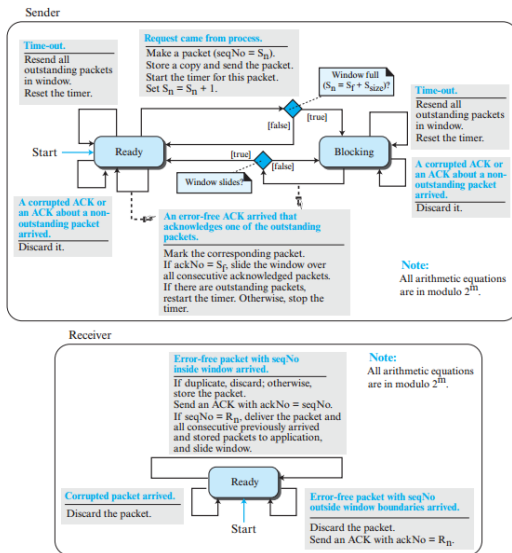
Receiver Operation

- 1 Wait for incoming packet.
- 2 Check packet for errors.
- 3 If packet is error-free and inside the receive window, store it in the corresponding buffer.
- 4 Send an ACK for that packet.
- 5 Deliver packets to the Application Layer only when all preceding packets have been received.
- 6 Slide the receive window whenever consecutive packets are delivered.

Key Point

Unlike Go-Back-N, the receiver stores out-of-order packets instead of discarding them.

Figure 3.34 FSMs for SR protocol



Selective Repeat FSM

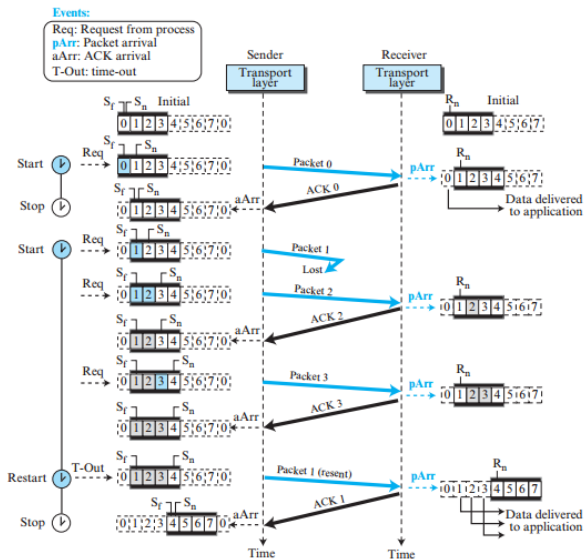
Sender

- Sends packets within the send window.
- Marks individual packets as acknowledged.
- Retransmits only timed-out packets.

Receiver

- Buffers out-of-order packets.
- Sends individual acknowledgments.
- Delivers packets only in sequence.

Figure 3.35 Flow diagram for Example 3.10



Window Size in Selective Repeat

Maximum Window Size

For Selective Repeat,

$$\text{Send Window Size} = \text{Receive Window Size} = 2^{m-1}$$

Example

If



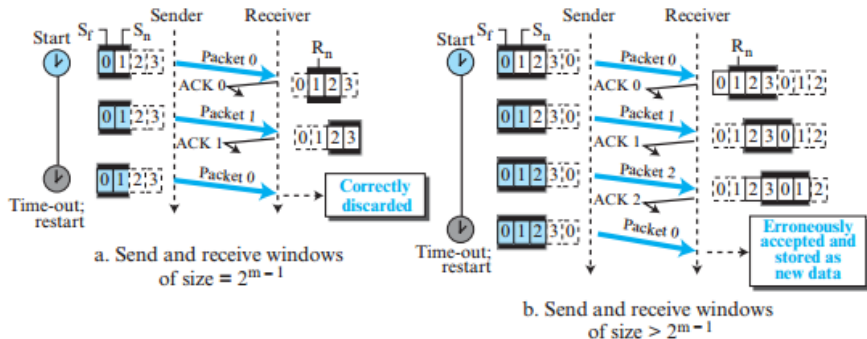
then

$$2^{m-1} = 2$$

Therefore,

- Send Window = 2
- Receive Window = 2

Figure 3.36 *Selective-Repeat, window size*



Why Window Size Must be 2^{m-1}

Case 1: Window Size = 2

- Lost ACKs do not create ambiguity.
- Duplicate packets are correctly discarded.

Case 2: Window Size = 3

- Duplicate packet falls inside the receive window.
- Receiver accepts duplicate as a new packet.
- Incorrect packet delivery occurs.

Important

To avoid ambiguity,

$$\text{Window Size} \leq 2^{m-1}$$

Selective Repeat – Summary

Features

- Sliding Window Protocol
- Flow Control
- Error Control
- Buffers out-of-order packets
- Individual ACKs
- Selective retransmission

Comparison with Go-Back-N

- Receiver window > 1
- Better bandwidth utilization
- Higher receiver complexity
- More buffering required
- Fewer retransmissions
- Higher efficiency

Key Takeaways

- Maximum Send Window = Maximum Receive Window = 2^{m-1}
- Receiver buffers out-of-order packets.
- Only erroneous packets are retransmitted.
- ACKs are individual, not cumulative.
- Provides better efficiency than Go-Back-N.

Bidirectional Protocols: Piggybacking

Unidirectional vs Bidirectional Protocols

The four protocols discussed earlier (Simple Protocol, Stop-and-Wait, Go-Back-N, and Selective Repeat) are **unidirectional**.

- Data packets travel in only one direction.
- Acknowledgments travel in the opposite direction.

Real Communication

- Client sends data to Server.
- Server also sends data to Client.
- Therefore, acknowledgments must flow in both directions.

Key Point

Real transport protocols are generally **bidirectional (full duplex)**.

Piggybacking

Definition

Piggybacking is a technique in which an acknowledgment is attached to a data packet traveling in the opposite direction.

Instead of sending

- one packet carrying data
- and another packet carrying ACK,

both are combined into a single packet whenever possible.

Advantages

- Reduces network overhead.
- Improves bandwidth utilization.
- Increases communication efficiency.

Working of Piggybacking

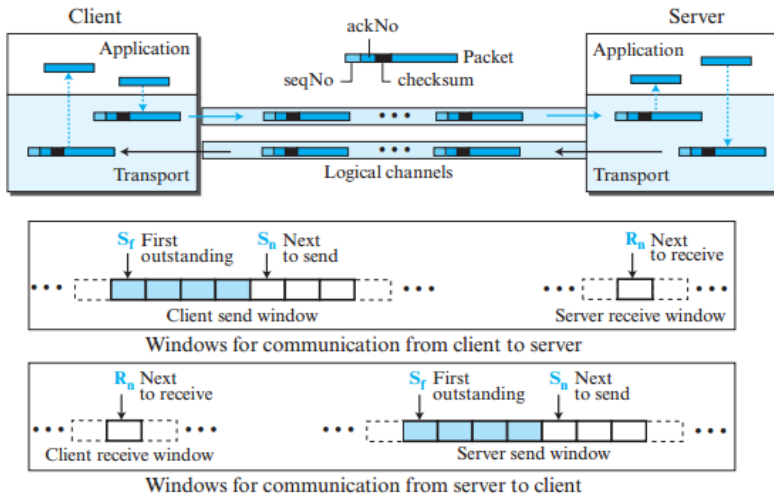
Client → Server

- Sends data packet.
- Includes ACK for packets received from Server.

Server → Client

- Sends data packet.
- Includes ACK for packets received from Client.

Figure 3.37 *Design of piggybacking in Go-Back-N*



Transport Layer in TCP/IP

The transport layer acts as an intermediary between

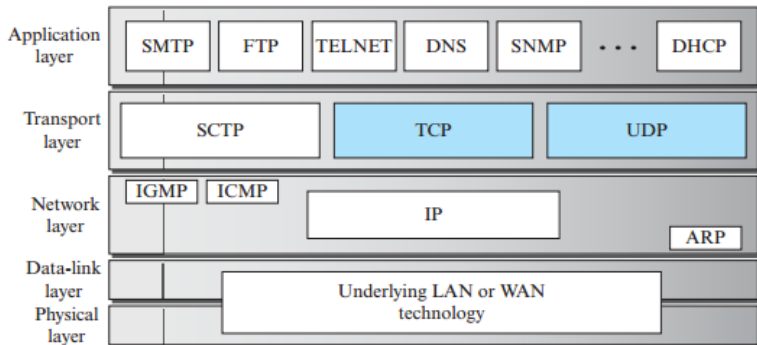
- Application Layer
- Network Layer

and provides **process-to-process communication**.

The Internet mainly uses two transport-layer protocols:

- User Datagram Protocol (UDP)
- Transmission Control Protocol (TCP)

Figure 3.38 *Position of transport-layer protocols in the TCP/IP protocol suite*



UDP and TCP Overview

UDP

- Connectionless
- Unreliable, Simple and Efficient
- Error control handled by Application Layer

TCP

- Connection-Oriented
- Reliable
- Provides Flow Control and Error Control

Transport-Layer Responsibility

Transport-layer protocols create **process-to-process communication** using **port numbers**.

Key Takeaways

- Bidirectional communication uses Piggybacking.
- UDP emphasizes speed and simplicity while TCP emphasizes reliability.
- Port numbers identify application processes.